CODE COVERAGE REPORT

Real-Time Chat Application Using Socket Programming

Mini Project Part-3, item 2(b)(iii)

Item	Details
Project	Real-Time Chat Application Using Socket Programming
Course	Software Engineering (UE23CS341A), PES University. Mini Project Part-3: Testing
Team	Kabir Raju G (Kabir7code) Keerthana K (KeerthanaK0304) Joshua (jjosh2005)
Repository	github.com/Kabir7code/Chat-application-Socket-Programming-
Test run	11 Nov 2026
Code tested	The complete Sprint 3 code: main on 11 Nov 2026 plus the Sprint 3 pull requests for #53, #54, #59, #60 and #61, which were being merged that day.

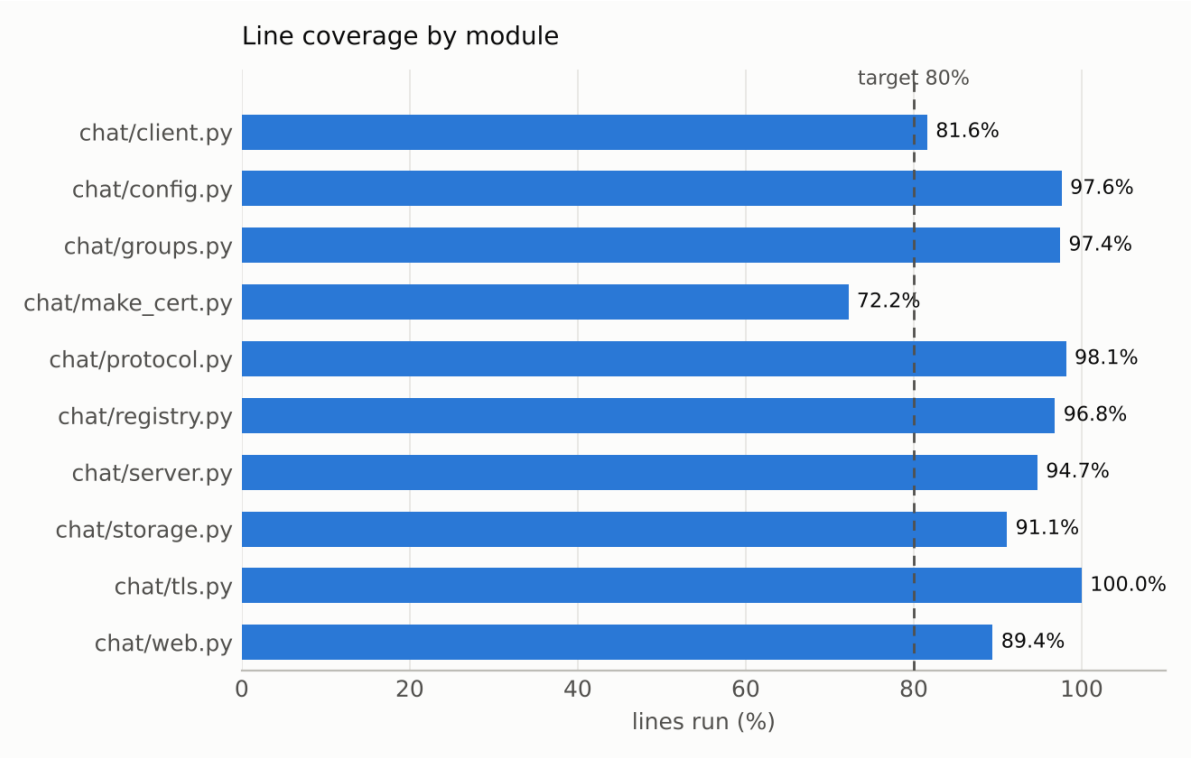
1. What was measured

Code (line) coverage is the share of executable lines in the chat package that the automated test suite runs at least once. It shows which parts of the code have no test at all.

- Tool: coverage.py 7.17 through pytest-cov, with branch measurement on (--cov-branch).
- Code measured: every module in chat/ (the tests themselves are not counted).
- Tests: the full automated suite, 338 tests, all passing. Programs started by the tests (the terminal client and the server run from the command line) are measured too (coverage subprocess patch).
- Run: 11 Nov 2026, Python 3.13. Code: The complete Sprint 3 code: main on 11 Nov 2026 plus the Sprint 3 pull requests for #53, #54, #59, #60 and #61, which were being merged that day.
- Target from the Test Plan: at least 80% of lines and 75% of branches.

2. Overall result

Measure	Result
Executable lines	1235
Lines run by the tests	1130
Lines never run	105
Line coverage	91.5% (target 80%: met)
Branch coverage (see the Branch Coverage Report)	86.1%
Combined (lines + branches, coverage.py's total)	90.4%



3. Coverage by module

Module	Role	Lines	Run	Missed	Coverage
chat/client.py	terminal client	136	111	25	81.6%
chat/config.py	settings and command-line options	42	41	1	97.6%
chat/groups.py	group chats	117	114	3	97.4%
chat/make_cert.py	certificate tool (run by hand)	36	26	10	72.2%
chat/protocol.py	message framing	54	53	1	98.1%
chat/registry.py	connected users	63	61	2	96.8%
chat/server.py	chat server	285	270	15	94.7%
chat/storage.py	SQLite message store	135	123	12	91.1%
chat/tls.py	TLS encryption	27	27	0	100.0%
chat/web.py	web UI gateway	340	304	36	89.4%
Total		1235	1130	105	91.5%

4. What is not covered, and why

Module	Lines never run	What they are
chat/client.py	85, 138-139, 143, 149, 156-161, 168-169, 171-172, 175, 180, 183-184, 188-191, 195-196	code that only runs in the interactive program: the background thread that prints incoming messages, the certificate and connection error exits of main(), Ctrl+C and 'connection lost' handling, and the end-of-input and retry paths of register(). The scripted test cases run these (UT_01, ST_02, IT_26, ST_17, ST_18), but they are not part of the coverage run.
chat/config.py	106	the server's TLS setup from certificate files, used only when the server is started with real certificate files.

Module	Lines never run	What they are
chat/groups.py	50, 108, 122	three error checks no automated test triggers: loading a saved group with no members, an admin adding someone who is already a member, and leaving a group you are not in.
chat/make_cert.py	37, 51, 53, 55-60, 64	the command-line part of the certificate tool (python -m chat.make_cert) and its 'openssl not found' error; the team runs it once by hand.
chat/protocol.py	79	the case where the connection closes between a frame's length header and its body.
chat/registry.py	47-48	an error from shutdown() on a socket that is already closed.
chat/server.py	107-114, 116, 131-132, 274, 298-299, 439	serve_forever() and the command-line start (run by ST_19 as a separate program, outside the coverage run), the accept loop ending when the listener closes, and two error returns in private messaging (text rejected by validation; a failure to queue a message for an offline user).
chat/storage.py	164-166, 206-208, 225-227, 238-240	the database-error branches of save_group_state, remember_user, queue_offline and take_offline. The message-saving error branch is covered, and ST_22 checks a failing database end to end.
chat/web.py	103, 159, 174, 186, 192, 196-198, 213-214, 270-272, 282-283, 358-359, 387-388, 409-410, 413, 438, 445-446, 448-449, 451, 453, 455, 457-460, 462, 466	main() (starting the gateway from the command line), HTTP error replies for malformed requests (missing Content-Length, a body that is not a JSON object, an unknown URL, a bad command line) and connection errors to the chat server (certificate rejected, server closed or not answering).

Most of what is missed is start-up code run from the command line, the certificate tool, and error branches for situations that are hard to cause on purpose (a database failure in one particular operation, a malformed HTTP request). The scripted test cases cover the start-up paths (ST_02, ST_19, IT_26) and a failing database (ST_22). A few user-facing error checks (leaving a group you are not in, adding an existing member, malformed web requests) have no automated test yet; adding them is the easiest way to raise coverage further.

5. How to reproduce

```
pip install -r requirements-dev.txt pytest-cov
pytest --cov=chat --cov-branch --cov-report=term-missing --cov-report=html
```

Attached in Documents/Test_Reports: coverage_html/index.html (open in a browser; every module with its missed lines in red and partly-taken branches in yellow), coverage.xml and coverage.json.